HAI930I - Systèmes multi-agents
Tournoi d'Axelrod : un LLM auto-adaptatif (MAPE-K) face à des stratégies déterministes

Dejean Robin, Hafdane Léo, Texier-Grailhes Killian

Novembre 2026

1 Introduction

Le *dilemme du prisonnier itéré* est le cadre classique pour étudier la coopération entre agents égoïstes : à chaque round, chacun coopère ou trahit (gains 3/3 si les deux coopèrent, 1/1 si les deux trahissent, 5/0 pour celui qui trahit un coopérateur). Trahir est la meilleure réponse immédiate, mais la coopération mutuelle rapporte plus à long terme, et c'est ce que le tournoi d'Axelrod a mis en évidence, avec des stratégies simples comme Tit For Tat qui battent des stratégies élaborées.

Notre projet y place un joueur piloté par un LLM (Gemini, via Google ADK) face à neuf stratégies déterministes classiques (bibliothèque Axelrod). À chaque round, le LLM joue selon une *stratégie de prompt* (Generous, Pragmatic, Merciless ou Peacemaker). Tous les `mapek_every` rounds, une boucle **MAPE-K** (*Monitor, Analyze, Plan, Execute, Knowledge*), issue du calcul autonome, peut changer cette stratégie. La question du sujet est : **un LLM auto-adaptatif peut-il remonter au score (« comeback ») quand il détecte qu'il joue mal ?**

Nos objectifs étaient (1) construire un banc d'essai reproductible, (2) mesurer l'effet de l'adaptation face à une référence à stratégie figée, (3) identifier les paramètres influents, et (4) comprendre pourquoi l'adaptation aide ou non.

2 Conception et implémentation

2.1 Architecture

Le projet est en Python 3.12, avec la bibliothèque Axelrod 4.14 pour les matchs et les adversaires, et Google ADK avec un modèle Gemini (par défaut `gemini-3.1-flash-lite`) pour le LLM. Le système comprend : un **moteur de match** (matrice de gains d'Axelrod, historique complet des coups et de la stratégie active) ; neuf **adversaires déterministes**, soit inconditionnels (Cooperator, Defector, Alternator), soit réactifs aux coups du LLM (Tit For Tat, Tit For 2 Tats, Suspicious Tit For Tat, Grudger, Prober, Win-Stay Lose-Shift) ; un **joueur LLM** qui reçoit à chaque round le prompt de sa stratégie active, son score, le taux de coopération adverse et les `history_shown` derniers coups, puis répond C ou D ; et la **boucle MAPE-K**. Le LLM ne connaît jamais le nom de l'adversaire : il doit le déduire de ses coups. Les agents n'ont aucune mémoire conversationnelle, tout le contexte passe dans le prompt.

Les quatre stratégies de prompt fournies sont : **Generous** (coopère par défaut, ne trahit qu'après 3 trahisons consécutives), **Pragmatic** (réciproque façon Tit For Tat, mais pardonne une trahison isolée après 5 coopérations), **Merciless** (trahit à chaque round) et **Peacemaker** (rompt les spirales de trahison mutuelle en proposant de coopérer, et avertit par un D ponctuel quand elle se fait exploiter). Cela rend les comportements lisibles et limite le choix de la boucle à quatre options. La stratégie de départ par défaut est Generous, volontairement naïve pour rendre un comeback visible. Côté robustesse, un coup illisible ou un appel en échec est remplacé par un coup de Tit For Tat, les appels sont relancés jusqu'à 3 fois, puis le programme bascule sur un autre modèle gratuit si le quota est épuisé.

2.2 Boucle MAPE-K

Tous les `mapek_every` rounds, la boucle s'exécute en cinq étapes. **Monitor** (Python) calcule, sur les `window` derniers rounds, les scores moyens du LLM et de l'adversaire, les compare à la fenêtre précédente et lève une alerte POOR si la moyenne est sous `poor_threshold` (1,5 par défaut), GOOD si elle est au moins de 2,5, OK sinon. **Analyze** (LLM, réponse JSON) profile l'adversaire (traître, coopérateur, type Tit For Tat, rancunier, alternateur…) et juge la performance par rapport à ce qui est atteignable contre lui. **Plan** (LLM, réponse

JSON) choisit la stratégie des prochains rounds à partir de l'analyse, du catalogue de stratégies, du bilan de chacune (rounds joués, points moyens) et des rounds restants. **Execute** (Python) remplace la stratégie active, un nom inconnu la laissant inchangée. **Knowledge** conserve le profil de l'adversaire, les analyses passées et le bilan par stratégie du match. Une boucle coûte 2 appels LLM en plus du coup de chaque round, soit $\text{turns} + 2 \times \lfloor (\text{turns} - 1)/\text{mapek_every} \rfloor$ appels par match (12 avec les réglages par défaut).

2.3 Choix de conception

La **référence figée** (`adaptive = non`) permet d'attribuer un gain à la boucle plutôt qu'à la stratégie de départ. Les **adversaires déterministes** (graine Axelrod fixée) rendent le LLM seule source d'aléa, d'où plusieurs répétitions par configuration. Le **comeback** est mesuré sur une fenêtre fixe de 5 rounds pour toutes les conditions, et les modes sont comparés **cellule par cellule** (adversaire × stratégie de départ), pour qu'une combinaison plus jouée ne pèse pas plus.

2.4 Organisation, tests et assistants IA

Chaîne d'expérimentation. Le *data generator* du programme joue le produit cartésien des paramètres choisis, chaque combinaison étant un match 1v1, et ajoute chaque match à un historique JSON Lines. Chaque membre a généré son propre historique, ce qui permet de répartir les appels d'API. Le script `sort_history.py` fusionne ces fichiers, range les matchs par condition de test (même température, mêmes rounds, même réglage MAPE-K, mêmes prompts) et retire les matchs rejoués à l'identique après une interruption. Le script `visualizations.py` produit ensuite les graphiques de chaque condition. Nous nous sommes également beaucoup aidés de LLM pour développer et mettre en place les différents scripts, afin de passer moins de temps sur l'implémentation et de pouvoir nous concentrer davantage sur l'analyse des résultats.

3 Scénarios expérimentaux et résultats

3.1 Scénarios et métriques

Nous avons étudié **786 matchs** en faisant varier les paramètres du tableau 1, le seuil POOR restant fixe à 1,5 point par round. Une *cellule* désigne une configuration. Trois scénarios : (1) adaptatif contre figé, pour le gain global de MAPE-K ; (2) balayage des paramètres de la boucle et du LLM ; (3) analyse des comebacks, c'est-à-dire des matchs mal commencés.

Paramètre	Valeurs	Rôle
`adaptive`	oui / non	boucle MAPE-K active ou stratégie de départ conservée (référence *figée*)
`mapek_every`	1, 5, 10	la boucle tourne tous les N rounds
`turns`	10, 20	nombre de rounds par match
`window`	1, 5, 10, 20	fenêtre glissante de l'étape Monitor (rounds)
`temperature`	0,1, 0,2, 0,9	température d'échantillonnage du LLM
`repetitions`	1, 5	nombre de matchs par combinaison
`stratégie de départ`	4	Generous, Pragmatic, Merciless, Peacemaker
`MAPE-K window`	5, 10, 20	rounds pris en compte par l'étape Plan
`adversaire`	9	Cooperator, Defector, Tit For Tat, Tit For 2 Tats, Suspicious TFT, Grudger, Alternator, Prober, Win-Stay Lose-Shift

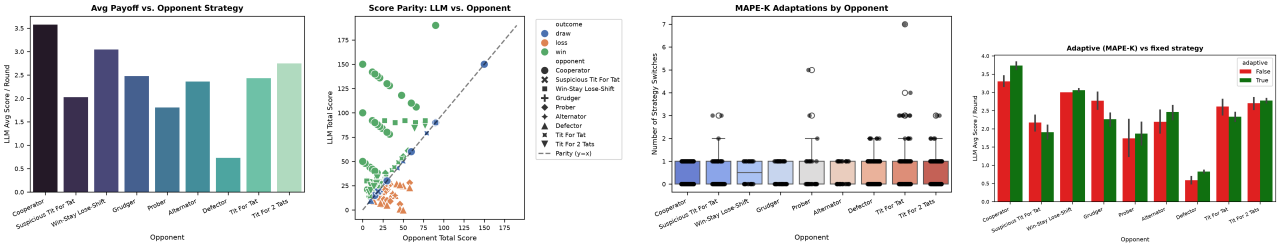
Table 1 – Paramètres que nous avons fait varier.

Métriques : **score moyen par round** (critère principal, entre 0 et 5) ; **issue** du match et **taux de coopération** ; **comeback** (moyenne des 5 premiers rounds sous 1,5 point par round et moyenne des 5 derniers supérieure d'au moins 0,5) ; **changements de stratégie** et **stratégie finale**.

3.2 Résultats

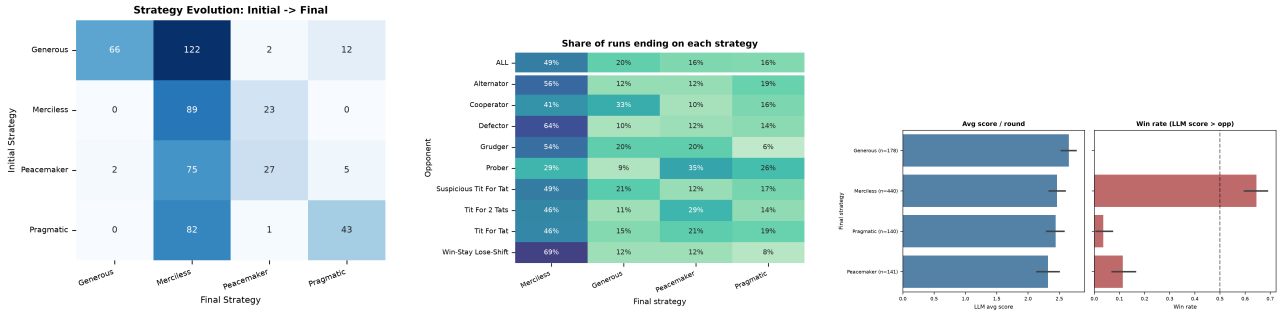
L'adversaire domine le score (fig. 1a,b) : le meilleur score est obtenu contre `cooperator`, le pire contre `defector`. Le nombre de changements de stratégie est corrélé à l'écart avec la version figée (c,d). Face aux adversaires réactifs comme `titfortat`, MAPE-K change souvent de stratégie sans améliorer le score, alors que contre les adversaires simples les changements sont rares et l'adaptation apporte souvent un gain. Nous supposons que la fenêtre d'observation est trop courte pour caractériser un adversaire réactif.

La boucle converge vers Merciless, quels que soient la stratégie de départ et l'adversaire (fig. 2). C'est la stratégie au meilleur taux de victoire, mais pas au meilleur score moyen : contre des adversaires coopératifs, `generous` fait fortement monter sa moyenne sans jamais gagner (défaite ou égalité).



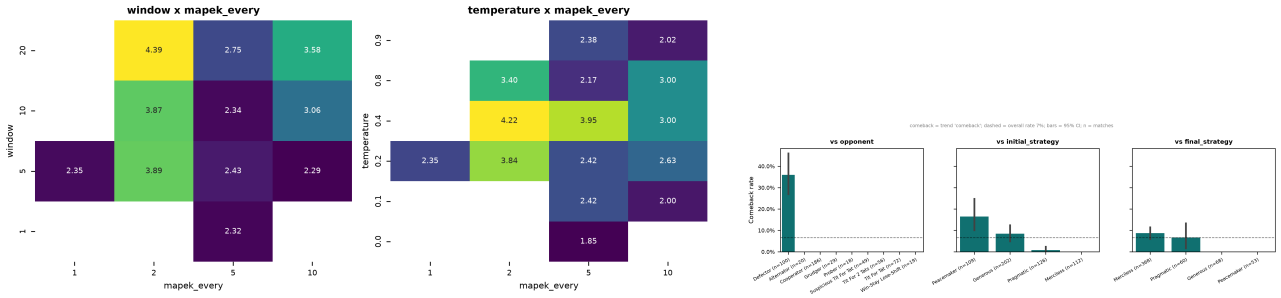
(a) Score moyen par adversaire. (b) Score LLM vs adversaire. (c) Changements de stratégie. (d) Adaptatif vs figé.

FIGURE 1 – Scores selon l'adversaire et activité de MAPE-K.



(a) Stratégie initiale → finale. (b) Stratégie finale par adversaire. (c) Score moyen et taux de victoire par stratégie finale.

FIGURE 2 – Stratégies sur lesquelles MAPE-K converge.



(a) Score moyen par paires de paramètres. (b) Comebacks selon l'adversaire et la stratégie.

FIGURE 3 – Influence des paramètres et comebacks.

Paramètres (fig. 3a) : les meilleurs résultats sont obtenus avec `mapek_every` = 2, une `window` plus grande et une température de 0,4, mais le nombre de parties par case est faible, donc ces tendances sont à confirmer. **Comebacks** (b) : ils sont presque uniquement possibles contre `Defector`, seul adversaire qui met d'abord le LLM en difficulté sans s'adapter ensuite. Les stratégies faibles (Peacemaker, Generous) remontent en basculant vers les fortes (Pragmatic, Merciless).

4 Discussion et conclusion

4.1 Principaux enseignements

Oui, un LLM auto-adaptatif peut remonter au score quand il joue mal. Contre Defector, 32 % des matchs mal commencés finissent en comeback avec MAPE-K, contre 5 % sans adaptation. Avec une boucle à chaque round, le score est de 0,91 point par round, contre 0,41 en figé, pour un maximum atteignable de 1 contre un traître inconditionnel.

Mais cela ne le rend pas meilleur en général : sur l'ensemble des adversaires, l'écart de score moyen est de 0,00 point par round. L'étape Plan a un **biais vers Merciless** (68 % des matchs adaptatifs y finissent) : le LLM trahit même quand tout va bien, ce qui casse la coopération avec les stratégies qui punissent (Grudger, Tit For Tat, Suspicious Tit For Tat). Il gagne plus de matchs (47 % contre 17 %) sans marquer plus de points, or dans l'esprit d'Axelrod seul le total compte. Monitor est un simple calcul en Python ; ce sont les étapes confiées au LLM, en particulier Plan, qui héritent de ses biais. Ce biais est cohérent avec la définition de Merciless, qui trahit à chaque round : c'est en pratique une stratégie « toujours trahir », qui ne peut jamais perdre un match (d'où son meilleur taux de victoire) mais empêche toute coopération mutuelle.

Le comeback n'apparaît que contre Defector, alors que nous l'attendions aussi contre Grudger et Suspicious

Tit For Tat en partant de Generous : contre un rancunier, une trahison initiale ne se rattrape pas. **Stratégies optimales observées** : Merciless contre les adversaires inconditionnels (Cooperator, Defector, Alternator) ; une stratégie coopérative (Pragmatic, Peacemaker ou Generous) contre ceux qui punissent la trahison. Une bonne boucle doit donc identifier la famille de l'adversaire et ne pas quitter une coopération mutuelle qui fonctionne.

4.2 Priorités, objectifs non atteints et limites

Nous avons privilégié une comparaison adaptatif/figé à configuration égale, puis la couverture de tous les adversaires et stratégies de départ, au prix de peu de répétitions (1 à 3 matchs par configurations). Chaque round étant un appel de modèle, le coût a limité le reste : le palier gratuit de Gemini est limité à environ 15 requêtes par minute, avec un quota journalier. Non atteints : **aucun tournoi avec boucle active** (le mode tournoi existe dans le programme, mais n'a pas été exploité) ; matchs de 20 rounds joués contre 3 adversaires seulement et sans référence figée ; pas de significativité statistique pour les écarts de quelques centièmes. Autres limites : le quota gratuit fait basculer automatiquement entre modèles Gemini, que nous avons supposé équivalents, ce qui n'est pas vérifié, le tout aurait dû être fait avec un seul modèle (`gemini-3.1-flash-lite` ou `gemma-4-26b-a4b-it`) ; les adversaires déterministes sont plus faciles à lire qu'un adversaire adaptatif.

Pistes : rendre le Plan plus prudent (ne pas quitter un état « GOOD », garder la coopération mutuelle comme objectif), allonger la fenêtre d'observation et les matchs avec une référence figée de même longueur, rejouer des tournois avec boucle active (par exemple tous les 5 rounds sur 30) avec davantage de répétitions, et comparer le choix libre du LLM à une règle explicite « identifier l'adversaire puis appliquer la meilleure stratégie connue ».

4.3 Bilan sur le cours HAI930I

Le projet a rassemblé plusieurs notions du cours : la **théorie des jeux** (aucune stratégie n'est optimale contre tous les adversaires), les **architectures adaptatives** (MAPE-K se transpose à tout système qui doit se surveiller et se reconfigurer), les **agents LLM** (comportements décrits en langage naturel, mais biais de décision, coût et variabilité) et la **méthode expérimentale** (référence, répétitions, prudence face aux écarts non significatifs), que nous réutiliserons pour évaluer tout système multi-agents ou d'apprentissage.

4.4 Conclusion

Un LLM auto-adaptatif peut remonter au score quand il joue mal, comme le montre le cas `Defector`, mais notre adaptation ne le rend pas meilleur en moyenne : la boucle détecte bien les mauvaises passes, et l'étape Plan, biaisée vers Merciless, détruit des coopérations qui fonctionnaient. La vraie question n'est pas « faut-il adapter ? » mais « vers quoi adapter, et quand s'arrêter ? ».